MÓDULO 3 · TRABAJO INDIVIDUAL

Ejercicio — Analizar antes de pedir

Resolución completa: análisis del código, tres problemas, prompt y justificación

ALUMNO
Crespo Santino

CURSO
6°2°

ESCUELA
E.T. N° 35 Ing. Eduardo Latzina

FUNCIÓN ANALIZADA
`queryTicketRows()`

ENDPOINT
`GET /tickets`

Resumen de la entrega

La función funciona hoy: trae 5 tickets, sin errores y con una sola persona. Por eso el ejercicio no busca un bug, sino responder dos preguntas: **qué va a pasar cuando el sistema crezca** y **qué decisiones quedaron tomadas sin que nadie las decidiera**. Este documento contiene:

- **Parte A — Lectura del código:** qué hace cada bloque y qué decisiones quedaron implícitas.
- **Parte B — Los tres problemas** ordenados del más grave al menos grave, cada uno con su escenario concreto y su detonante.
- **Parte C — Otros hallazgos** de menor peso y un mapa de qué se rompe y cuándo.
- **Parte D — El prompt** que le escribiría a la IA (sin código arreglado, como pide la consigna).
- **Parte E — La justificación** del prompt: contexto, límites y decisiones propias.
- **Parte F — Autoevaluación** contra los criterios de corrección.

Los tres problemas de un vistazo

PROBLEMA 1

SQL como texto

Sin parámetros: se rompe con un apóstrofe y permite inyección apenas haya formularios.

PROBLEMA 2

JSON cortado

El límite `-y 8000` trunca el resultado y el endpoint devuelve error 500 al crecer los datos.

PROBLEMA 3

Todo, cada vez

Toda la tabla, un proceso y una conexión nuevos por request: lento y frágil con volumen.

Parte A — Lectura del código

Antes de buscar problemas conviene entender qué hace el código realmente. Lectura bloque por bloque:

Bloque	Qué hace	Qué supone (sin decirlo)
`dotenv.config({path: ../../.env})`	Carga variables de entorno al importar el módulo, con una ruta relativa a la ubicación del archivo.	Que el archivo nunca se mueva de carpeta y que siempre exista un `.env`.
`promisify(execFile)`	Convierte la ejecución de un programa externo en una promesa.	Que el programa externo exista en el equipo que ejecuta el backend.

Bloque	Qué hace	Qué supone (sin decirlo)
`-S servidor -d base -E`	Conecta con autenticación integrada de Windows (sin usuario ni contraseña).	Que el backend corra en Windows, con un usuario que tenga permisos en la base.
`-y 8000 -h -1`	Ancho máximo de columna de 8000 caracteres y sin encabezados en la salida.	Que el resultado nunca supere 8000 caracteres.
`SELECT * FROM Tickets FOR JSON PATH`	Pide todas las columnas de todas las filas, ya armadas como un único texto JSON.	Que la tabla siempre sea chica y que todas las columnas sean públicas.
`maxBuffer = 10 MB`	Tope de bytes que Node acepta de la salida del proceso.	Que 10 MB alcancen para siempre.
`stdout.trim() -> JSON.parse`	Toma la salida de texto y la convierte en objetos.	Que la salida contenga solo JSON válido y completo (sin errores ni cortes).
`return [] si está vacío`	Devuelve lista vacía si no hubo salida.	Que "sin salida" significa "sin tickets" y no "algo falló".

Decisiones que quedaron tomadas sin que nadie las decidiera

Esta es la pista central del ejercicio. Ninguna de estas decisiones fue discutida: aparecieron porque era lo más rápido para que anduviera el primer día.

Decisión implícita	Cómo quedó	Por qué importa cuando el sistema crezca
Cómo hablamos con la base	Ejecutando un programa de consola en vez de usar un driver	No hay parámetros, conexiones reutilizables ni manejo de errores estructurado.
Cómo nos autenticamos	Autenticación de Windows (`-E`)	Ata el backend a Windows; complica Docker, Linux y la nube.
Cuántos datos traemos	Todos (sin `WHERE`, sin paginación)	El costo crece en proporción directa a la cantidad de tickets.
Qué columnas exponemos	Todas (`SELECT *`)	Cualquier columna nueva pasa a ser pública sin que nadie lo decida.
Qué tamaño máximo tiene un resultado	8000 caracteres y 10 MB, por casualidad	Son límites escondidos que fallan recién con datos reales.
Qué es un error	Un resultado vacío se trata como "sin tickets"	Los fallos pueden pasar desapercibidos y mostrarse como pantalla vacía.
Cómo se escribirá SQL en el futuro	Como texto dentro de un string	Es el camino más corto hacia la inyección SQL al agregar el formulario.

Parte B — Los tres problemas (del más grave al menos grave)

Criterio de orden. Ordené por tres preguntas: (1) *¿qué tan grave es el daño?* (pérdida o exposición de datos pesa más que una caída temporal); (2) *¿qué tan probable es?* (¿ya está en el plan del proyecto?); (3) *¿qué tan fácil es darse cuenta?* (un fallo silencioso es peor que uno ruidoso).

#	Problema	Daño	Probabilidad	Detectabilidad
1	SQL como texto, sin parámetros	Muy alto (datos y seguridad)	Muy alta: el formulario ya está planificado	Baja: funciona hasta que alguien escribe algo "raro"
2	Corte del JSON en 8000 caracteres	Alto (el endpoint deja de funcionar)	Alta: ocurre solo al crecer los datos	Media: aparece como error 500 sin causa obvia
3	Todo en cada request + un proceso por request	Medio (lentitud, caídas bajo carga)	Media: depende de cuántos usuarios y tickets	Alta: se nota al volverse lento

Problema 1 | El SQL viaja como texto por `sqlcmd`, sin parámetros

Dónde está en el código. La consulta se entrega como un string en el argumento `-Q` . `sqlcmd` no ofrece consultas parametrizadas: todo lo que se quiera enviar a la base tiene que estar escrito dentro de ese texto.

Por qué hoy no pasa nada. La consulta actual es fija y no incluye datos del usuario. El riesgo está en la *dirección* que el código obliga a tomar cuando se agregue crear y editar: la única forma de meter el título del formulario es pegarlo dentro del string.

Escenarios concretos

Escenario	Qué escribe la persona	Qué pasa
A. Apóstrofe común	Título: `No anda el login de O'Brien`	La comilla simple cierra el texto SQL antes de tiempo. El INSERT falla con error de sintaxis o guarda algo distinto a lo escrito. Cualquier apellido con apóstrofe, o un texto como "don't", rompe la carga.
B. Inyección SQL	Título: `x'); DROP TABLE Tickets; --`	El texto se ejecuta como comandos SQL. Se puede borrar la tabla, leer otras tablas o modificar datos de otras personas.
C. Edición masiva	Descripción con `'; UPDATE Tickets SET Status='Cerrado'; --`	Se cierran todos los tickets de todos los usuarios con una sola edición.
D. Cambio de entorno	(sin input del usuario)	El backend pasa a correr en Docker/Linux o en un servidor sin `sqlcmd` instalado, o sin sesión de Windows: `GET /tickets` falla porque depende de `-E` y del binario.

Por qué es el más grave. Es el único de los tres que puede causar *pérdida o filtración de datos*, no solo lentitud. Además es el más probable (el formulario ya está en el plan) y el más silencioso: anda perfecto en las pruebas con textos "normales".

Qué decisión hay que tomar. No es "escapar las comillas" (eso es un parche frágil), sino cambiar el mecanismo de acceso a uno que separe la consulta de los datos (consultas parametrizadas).

Problema 2 | El límite `-y 8000` corta el JSON y `JSON.parse` falla

Dónde está en el código. `FOR JSON PATH` devuelve *todas las filas como un único texto*. La opción `-y 8000` limita el ancho de ese texto. Si el JSON supera el límite, `sqlcmd` lo corta sin avisar y lo que llega a Node es un JSON incompleto.

Escenarios concretos

Escenario	Qué pasa en la práctica	Síntoma para el usuario
A. Crecimiento normal	Cada ticket ocupa unos 80 a 100 caracteres hoy; con descripciones reales de 300 a 500 caracteres, unos 15 a 25 tickets ya superan los 8000.	La pantalla de tickets deja de cargar un día cualquiera, sin que nadie haya tocado el código.
B. Un solo ticket grande	Alguien pega un mail o un log largo en la descripción de un ticket.	Un único registro rompe el listado de *todos* los tickets para *todas* las personas.
C. Caracteres especiales	Las comillas, barras y saltos de línea se escriben con secuencias extra (por ejemplo `\"` o `\n`) y el JSON crece más de lo que se ve a simple vista.	El límite se alcanza antes de lo calculado.
D. Diagnóstico difícil	El error que se ve es `SyntaxError: Unexpected end of JSON input`, que no menciona el límite ni la base.	Se pierde tiempo buscando en el lugar equivocado.

Por qué ocupa el segundo lugar. Garantiza una caída del endpoint principal, pero no daña datos ni expone nada. Es un fallo *seguro de ocurrir*: no depende de que alguien sea malicioso ni de la mala suerte, solo de que el sistema se use.

Qué decisión hay que tomar. No subir el número (mover el límite solo posterga la falla), sino dejar de depender de convertir todo en un texto único que hay que reconstruir: que la base devuelva filas y el código las arme.

Problema 3 | Trae toda la tabla en cada request y lanza un proceso nuevo cada vez

Dónde está en el código. `SELECT * FROM Tickets` no tiene `WHERE` ni `TOP/OFFSET`; y cada llamada a `GET /tickets` ejecuta `execFile('sqlcmd', ...)`, que crea un proceso del sistema operativo y abre una conexión nueva a la base.

Escenarios concretos

Escenario	Qué pasa	Consecuencia
A. Muchos tickets	Con 20.000 tickets cada GET descarga y convierte toda la tabla en memoria.	Respuestas de varios segundos y mucha memoria usada por request. Al pasar los 10 MB de `maxBuffer`, Node corta el proceso hijo y el endpoint falla con un error de desbordamiento.
B. Usuarios simultáneos	20 personas abren la pantalla a la vez: son 20 procesos `sqlcmd` y 20 conexiones nuevas al mismo tiempo.	Cada proceso tarda en arrancar, consume memoria y la base recibe 20 logins. El servidor se degrada y las respuestas se encolan.
C. Refrescos frecuentes	La pantalla se actualiza cada pocos segundos o alguien mantiene varias pestañas abiertas.	El costo se multiplica: por cada pestaña, un proceso nuevo y la tabla completa.
D. Columnas nuevas	Se agrega una columna `NotasInternas` o `EmailReportante` a la tabla.	Con `SELECT *` esa columna llega al cliente sin que nadie la haya aprobado: una filtración de datos por omisión.

Por qué ocupa el tercer lugar. Es el problema más "visible": se nota como lentitud antes de convertirse en caída, lo que da margen para reaccionar. Su daño es de rendimiento; la excepción es el punto D, que se acerca a una cuestión de seguridad.

Qué decisión hay que tomar. Cuántos tickets devuelve un pedido (paginación con un máximo), qué columnas son públicas y cómo se comparten las conexiones (un pool creado una sola vez).

Parte C — Otros hallazgos y mapa de crecimiento

Hallazgos de menor peso

No entran entre los tres principales (la consigna pide tres), pero conviene dejarlos anotados porque afectan el diseño del arreglo:

Hallazgo	Situación concreta	Gravedad
Errores de SQL que no se notan	`sqlcmd` sin la opción `-b` puede terminar con código de salida 0 aunque la consulta haya fallado, y escribe el mensaje de error en la salida normal. Node intenta hacer `JSON.parse` de un mensaje de error.	Media
Vacío tratado como "sin tickets"	Si `sqlcmd` falla y no imprime nada, la función devuelve `[]`. El usuario ve una lista vacía y cree que no hay tickets, cuando en realidad hay un error.	Media
`DB_DATABASE` vacío sin validar	Solo se valida `DB_SERVER`. Si falta `DB_DATABASE`, `sqlcmd` usa la base por defecto del login y se puede estar leyendo (o escribiendo) la base equivocada sin darse cuenta.	Media
Sin tiempo máximo (timeout)	Si la base no responde, el proceso hijo queda colgado y el request nunca termina; los procesos se acumulan.	Media
Contraseña visible al pasar a usuario/clave	Si se reemplaza `-E` por `-U/-P` en `sqlcmd`, la contraseña viaja como argumento del proceso y puede verse en la lista de procesos del sistema o en mensajes de error.	Media
Acentos y caracteres especiales	`sqlcmd` usa por defecto la página de códigos de la consola de Windows; textos con ñ, tildes o emojis pueden llegar alterados al leerlos como UTF-8 en Node.	Baja a media
Ruta de `.env` frágil	La ruta relativa a `__dirname` deja de funcionar si se mueve el archivo o cambia cómo se compila el proyecto.	Baja
Sin pruebas automáticas	Nada verifica que el formato de salida siga siendo el mismo al cambiar la implementación.	Baja

Mapa de crecimiento: qué se rompe y cuándo

Momento	Situación	Qué pasa con la función actual
Hoy	5 tickets, 1 usuario	Todo anda. Es la situación en la que las decisiones implícitas son invisibles.
Cuando llegue el formulario	Se agregan crear y editar con texto libre	Problema 1: apóstrofes rompen el guardado; aparece el riesgo de inyección.
Cuando los tickets tengan descripciones reales	20 a 100 tickets	Problema 2: el JSON supera 8000 caracteres y el listado devuelve error 500.
Cuando lo use un equipo	10 a 20 personas simultáneas	Problema 3: decenas de procesos y conexiones en paralelo; respuestas lentas.
Cuando acumule historial	Miles de tickets	Problema 3: memoria, tiempos altos y corte por `maxBuffer`.
Cuando se despliegue	Docker, Linux o nube	Problema 1 (parte D): el backend no puede ejecutar `sqlcmd` ni usar autenticación de Windows.

Parte D — El prompt

Siguiendo la consigna, no entrego el código arreglado: entrego el prompt que le escribiría a la IA. Está dividido en bloques con un rótulo para que la IA pueda distinguir el *contexto*, el *objetivo*, las *decisiones ya tomadas*, los *límites* y el *formato de respuesta*. En el PDF los bloques se ven separados; al usarlo se pegan juntos, en este orden.

1) CONTEXTO

```
Trabajo en un backend Node.js + TypeScript. La función queryTicketRows() (archivo
src/db/tickets.ts) atiende GET /tickets y trae los tickets de SQL Server. Hoy lo
hace ejecutando el programa sqlcmd con child_process.execFile y la consulta "SELECT
* FROM Tickets FOR JSON PATH", y después hace JSON.parse de la salida.

Situación actual: 5 tickets con títulos cortos y 1 persona usando el sistema.
Funciona sin errores.
Situación esperada en unos 3 meses: unos 2.000 tickets con descripciones de texto
libre (de hasta varios miles de caracteres) y unas 20 personas usando el sistema a
la vez. Además vamos a agregar crear y editar tickets desde un formulario, por lo
que el texto de los usuarios va a llegar a la base.
El backend hoy corre en Windows, pero queremos poder desplegarlo en Docker/Linux.
```

2) PROBLEMAS A RESOLVER (en este orden de prioridad)

```
1. sqlcmd no admite consultas parametrizadas. Cuando se guarde texto libre, un
   apóstrofe (por ejemplo "O'Brien") rompe la consulta y un texto malicioso permite
   SQL injection. Además depende de tener el binario sqlcmd y de la autenticación de
    Windows (-E).
2. La opción -y 8000 corta la salida de FOR JSON cuando supera 8000 caracteres y
   JSON.parse falla con SyntaxError; pasaría con unas pocas decenas de tickets con
   descripciones reales o con un solo ticket largo.
3. SELECT * trae toda la tabla en cada request, sin paginación, y cada request crea
   un proceso y una conexión nuevos. Con 2.000 tickets y 20 usuarios simultáneos se
   vuelve lento y puede pasar el maxBuffer de 10 MB. Además SELECT * expondría
   cualquier columna que se agregue en el futuro.
```

3) DECISIONES QUE YA TOMÉ (no las cambies ni las discutas)

```
- Reemplazar sqlcmd por el paquete npm "mssql" (driver tedious) usando UN solo pool
   de conexiones compartido, creado una única vez y reutilizado entre requests. No
   agregues ninguna otra dependencia (nada de ORM, ni Prisma, ni TypeORM, ni knex).
- Autenticación con usuario y contraseña (variables DB_USER y DB_PASSWORD en .env).
   Se abandona -E. Hay que validar al iniciar que existan DB_SERVER, DB_DATABASE,
   DB_USER y DB_PASSWORD; si falta alguna, fallar con un mensaje que nombre
   exactamente la variable faltante.
- Todas las consultas usan parámetros tipados (request.input con tipo explícito).
   Está prohibido armar SQL concatenando o interpolando strings con datos externos.
- Paginación con parámetros limit y offset: limit por defecto 50 y máximo 200;
   offset por defecto 0; orden fijo ORDER BY Id DESC (necesario para que la
   paginación sea estable). Si limit u offset no son enteros no negativos, rechazar
   con un error claro en lugar de corregirlos en silencio.
- Seleccionar columnas explícitas: Id, Title, Description, Status, CreatedAt. Si la
   tabla tiene otras columnas, NO las incluyas.
- Timeout de consulta de 10 segundos.
- Si la base devuelve un error, propagarlo como error (no devolver una lista vacía).
    "Lista vacía" solo significa que no hay tickets.
```

4) QUÉ NO TOCAR

```
- El nombre de la función, su ubicación y su contrato: sigue siendo una función
   async que devuelve una lista de tickets (Promise<unknown[]> o un tipo más preciso
    de esas 5 columnas, solo si no obliga a cambiar a quien la llama).
- La ruta GET /tickets, los controllers, el resto de los archivos y cualquier otra
   parte del proyecto. Solo se modifica src/db/tickets.ts, más el cambio mínimo en
   .env.example para documentar las variables nuevas.
- La carga de dotenv y el formato de los mensajes de error existentes.
- No agregues: caché, reintentos automáticos, logging, ni las funciones de crear o
   editar tickets. Eso se hará después, en otro pedido.
```

5) CÓMO QUIERO LA RESPUESTA

```
Paso 1 (antes de escribir código): resumime en 5 líneas como máximo tu plan y decime
 si algo de lo que te pedí es contradictorio, ambiguo o imposible. Si necesitás
asumir algo, preguntame en vez de asumirlo y esperá mi respuesta.
Paso 2: el código completo solo de src/db/tickets.ts, con comentarios breves en los
cambios importantes.
Paso 3: una lista con (a) qué cambió, (b) qué dejaste igual a propósito y (c) qué
riesgos o pendientes quedan fuera de este cambio.
Paso 4: decime cómo probarlo manualmente: qué pedidos hacer con limit y offset, y
qué debería verse si falta una variable de entorno o si la base no responde.
```

Nota sobre el prompt
El prompt no empieza con "mejorá este código": empieza con el contexto y el futuro del sistema. No incluye el código arreglado; solo describe problemas, decisiones, límites y formato de salida.

Parte E — Justificación del prompt

1. Qué contexto puse adentro y por qué

Contexto incluido	Por qué lo puse
Cómo funciona hoy la función (archivo, endpoint, sqlcmd, FOR JSON)	La IA no ve mi repositorio: sin esto tendría que adivinar la arquitectura y podría "arreglar" cosas que no existen o reescribir todo.
Cifras actuales y esperadas (5 -> 2.000 tickets; 1 -> 20 usuarios)	Con el código funcionando, la IA tiende a dejarlo igual o a hacer cambios cosméticos. Los números del futuro son lo que vuelve reales los problemas y permiten elegir un límite de paginación razonable.
Que se va a agregar el formulario con texto libre	Es lo que convierte a la falta de parámetros en un problema. Sin esto, el problema 1 parecería teórico.
Que hoy corre en Windows pero se quiere desplegar en Docker/Linux	Justifica sacar `-E` y `sqlcmd`. Sin este dato, la IA podría mantener la autenticación de Windows "porque funciona".
Los tres problemas con su escenario y orden de prioridad	Le dice qué importa más si tiene que hacer concesiones, y la obliga a resolver causas, no síntomas (por ejemplo, no "subir el 8000 a 80000").

2. Qué decidí yo en lugar de dejárselo a la IA

La consigna pregunta exactamente esto. Para cada decisión, expliqué qué habría pasado si se la dejaba a la IA:

Decisión que tomé yo	Qué podría haber elegido la IA	Por qué la tomé yo
Driver: `mssql` con un único pool	Un ORM completo (Prisma, TypeORM), o `msnodesqlv8` para mantener Windows Auth, o seguir con `sqlcmd` "parcheado".	Un ORM es un cambio enorme para una función de 30 líneas. Mantener Windows Auth ata el proyecto a Windows. Es una decisión de arquitectura que tengo que poder defender.
Autenticación por usuario y contraseña desde `.env`	Dejar `-E` o inventar otro esquema (tokens, Azure AD).	Es una decisión de seguridad y de despliegue. Ni la IA ni el código saben cómo vamos a desplegar.
Consultas siempre parametrizadas y tipadas	Escapar comillas a mano, o mezclar parámetros con concatenación.	Escapar es un parche frágil. La regla tiene que ser absoluta para que sea fácil de revisar.
Paginación con valores concretos (50 / 200 / orden por `Id DESC`)	Sin límite máximo, con otros valores, o paginación por cursor.	Son decisiones de producto: cuánto se muestra por pantalla y en qué orden. El máximo protege a la base de pedidos enormes.
Rechazar parámetros inválidos en lugar de corregirlos	Corregirlos en silencio (por ejemplo, limit negativo pasa a 50).	Corregir en silencio esconde errores del que llama; prefiero que falle claro.
Columnas explícitas (Id, Title, Description, Status, CreatedAt)	Mantener `SELECT *` o elegir las columnas que "parezcan" importantes.	Qué datos se exponen es una decisión mía, no algo que se deduzca del código.
Errores propagados; lista vacía solo si no hay tickets	Devolver `[]` ante cualquier fallo "para que no rompa".	Un sistema que oculta errores es más difícil de mantener que uno que avisa.
Timeout de 10 segundos	Sin timeout, o uno arbitrario.	Es una decisión de experiencia de usuario: cuánto se espera antes de fallar.

3. Qué límites le puse y por qué

- **Alcance de archivos:** solo `src/db/tickets.ts` (más un cambio mínimo en `.env.example`). Una IA servicial tiende a "aprovechar" para reordenar controllers, renombrar cosas o cambiar formatos, lo que rompe lo que hoy anda y hace imposible revisar el cambio.
- **Contrato de la función:** mismo nombre, misma ubicación, mismo tipo de resultado. Así quien la llama no se entera del cambio.
- **Prohibición de dependencias extra:** cada librería nueva es superficie de mantenimiento y de seguridad.
- **Prohibición de extras (caché, reintentos, logging, crear/editar):** cada extra agranda el cambio. Prefiero cambios chicos, revisables y por etapas; el formulario será otro pedido.
- **Lo que no cambia de la carga de entorno y de los mensajes de error:** son convenciones existentes que otras partes pueden estar usando.

4. Cómo le pedí que trabaje

- **Plan antes del código** y **preguntar en lugar de asumir:** para que las decisiones queden visibles y no escondidas dentro del código generado. Si mi pedido tuviera una contradicción, prefiero enterarme antes.
- **Solo el archivo afectado:** para poder compararlo con el original y revisarlo completo.
- **Lista de "qué dejaste igual a propósito":** me da un criterio para revisar el resultado. Si cambió algo que figura en "no tocar", lo veo de inmediato.
- **Cómo probarlo:** un arreglo sin forma de verificarlo es solo una promesa.

5. Qué dejé abierto a propósito

No lo decidí todo: dejé a la IA los detalles de implementación que no cambian el comportamiento que me importa: cómo estructurar el pool, qué tipo exacto de TypeScript devolver, cómo redactar los mensajes nuevos y los comentarios. Decidir esas cosas por ella sería microgestionar y no aportaría nada.

6. Qué revisaría del resultado

- Que no haya ninguna consulta armada con concatenación o interpolación de strings.
- Que el pool se cree una sola vez (no una conexión por llamada).
- Que no se hayan tocado otros archivos ni la firma de la función.
- Que un `limit` mayor a 200 o un valor negativo sea rechazado.
- Que la falta de cualquiera de las cuatro variables dé un mensaje con su nombre.
- Que un error de la base se propague y no se convierta en `[]`.

Parte F — Autoevaluación con los criterios de corrección

Criterio de la consigna	Cómo lo cumple esta entrega
¿El prompt dice qué camino tomar, o deja que la IA elija por mí?	Dice el camino en el bloque 3: `mssql`, pool único, parámetros tipados, autenticación por variables, paginación con valores concretos, columnas explícitas, timeout y manejo de errores.
¿Dice qué no hay que tocar?	Bloque 4: nombre y contrato de la función, ruta, controllers, otros archivos, dotenv, mensajes de error, y una lista de cosas que no hay que agregar.
¿Los tres problemas son situaciones concretas o adjetivos?	Cada uno incluye escenarios con ejemplos reales (apóstrofe en "O'Brien", descripción de mail pegada, 20 usuarios abriendo la pantalla) y el detonante que los activa. No uso "desprolijo" ni "se puede mejorar".

Cumplimiento de las reglas

Regla	Estado
Primero analizar solo; usar IA recién en el punto 2	Cumplido: el análisis de las partes A a C es independiente del prompt.
No entregar el código arreglado	Cumplido: solo se entrega el prompt.
El prompt no empieza con "mejorá este código"	Cumplido: empieza con el contexto y el escenario futuro.
Tres problemas ordenados de más a menos grave	Cumplido (Parte B, con el criterio de orden explicado).
Justificación: contexto, límites y decisiones propias	Cumplido (Parte E, secciones 1 a 3).

Glosario breve

Término	Significado
SQL injection	Ataque en el que un texto escrito por el usuario se interpreta como comandos SQL porque se pegó dentro de la consulta.
Consulta parametrizada	Consulta en la que el texto SQL y los datos viajan por separado; la base nunca interpreta los datos como comandos.
Pool de conexiones	Conjunto de conexiones a la base que se crean una vez y se reutilizan entre pedidos.
Paginación	Devolver los datos de a tandas (por ejemplo, 50 por vez) en lugar de todos juntos.
`FOR JSON PATH`	Cláusula de SQL Server que devuelve el resultado de una consulta como texto JSON.
`maxBuffer`	Límite de bytes que Node acepta de la salida de un proceso hijo antes de cancelarlo.
Timeout	Tiempo máximo de espera antes de darse por vencido y devolver un error.